



BIT2083 FUNDAMENTAL OF COMPUTATIONAL THINKING: PHYTHON

FINAL PROJECT - GROUP (40%)

ALL SECTIONS

FACULTY OF INFORMATION TECHNOLOGY

CITY UNIVERSITY MALAYSIA

CYBERJAYA CAMPUS

Github repository : https://github.com/babaset/University_green_challenge

Prepared by

Full name	Student ID	Class code	Program	NRIC / Passport
Omar Algaafari	202505010463	BIT2083	BCSSE	10646149
Babasit Mohammed Ahmed Saleh	202505010085	BIT1054	BCSSE	13097825
Kisrane Amine	202505010489	BIT2083	BCSSE	307528754
Ghaith Aldain Gamil Hamid Melhi Qasem	202501010660	BIT2083	BCSSE	11772197
Basel Tajeldeeen Marsi Idris	202504010013	BIT1123	BCSSE	P10532763

Table of Contents

Prepared by	1
Executive Summary	4
1. Introduction and SDG Background	4
1.1 SDG 11 Alignment	4
1.2 Project Overview	5
Student login screen and University Green Challenge value proposition	5
2. Problem Statement	5
2.1 Real-World Problem	5
2.2 Causes and Effects	6
3. System Objectives and Scope	6
3.1 Main Objective	6
3.2 Specific Objectives	6
3.4 Quality Requirements	7
4. Computational Thinking Approach	7
4.1 Decomposition	7
4.2 Pattern Recognition	8
4.4 Algorithmic Thinking	8
5. Application Design	10
5.1 Layered Architecture	10
5.2 UML Class Diagram	11
5.3 Data Model	12
5.4 Eco-Action Scoring Design	13
6. Implementation Details	13
6.1 Technology Stack	13
6.2 Repository and Module Structure	14
6.3 Codebase Metrics and Modularity	15
6.4 Variables, Operators, Conditions and Loops	15
6.5 Input Validation and Error Handling	16
6.6 User Experience and Role-Based Navigation	16
Administrator login view with role selection	17
7. Testing and Sample Outputs	18
7.1 Testing Strategy	18
7.2 Functional Test Results	18
7.3 Sample Output: Student Submission	19

7.4 Sample Output: Administrator Review.....	20
Claim approval confirmation with awarded points	21
7.5 Sample Output: Leaderboard and Analytics	21
Leaderboard ranked by points and trust progress.....	22
8. Discussion and Limitations.....	23
8.1 Achievement of Objectives.....	23
8.2 Strengths.....	23
8.3 Current Limitations	23
8.4 Future Improvements.....	23
9. Conclusion	24
References.....	24

Executive Summary

University Green Challenge is a Python desktop application to make it easy, visible and rewarding for people to live more sustainably in a university community. Eco-actions are verified and students submit pictures of their actions, receive points when approved by administration and can compare their actions through a transparent leaderboard. Administrators register students, monitor evidence, track claim statuses and participation via the Analytics dashboard.

The project also contributes to SDG 11 by promoting behavioural changes focusing on sustainable mobility, waste reduction, efficient resource utilization and improvement of shared space. It has the highest linkage to Target 11.2 on sustainable transport and Target 11.6 on reducing the environmental impact of cities through waste management. The application adopts computational thinking by breaking things apart, by spotting patterns, by abstracting, by designing algorithms, and its code is split between the application's interface, controller, business logic, data models and persistence logic.

Implementation evidence includes role-based login, a student claim workflow, evidence hashing, trustscore management, a ranked leaderboard, and JSON persistence and live analytics. Temporary test data was used with eight independent checks and all eight passed. The included screenshots also show the full student-to-administrator approval process while not changing the project source code.

Project value proposition: Make sustainable living easy and rewarding, by making every eco-action a verified progress and points and a positive competition.

1. Introduction and SDG Background

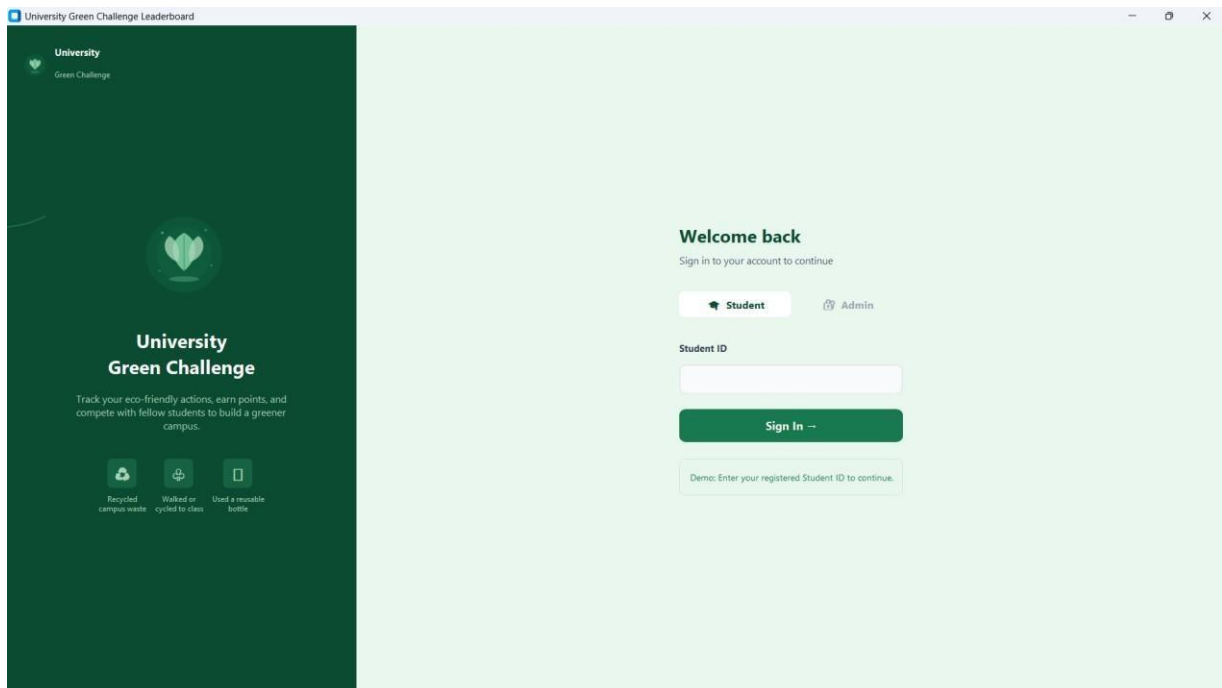
1.1 SDG 11 Alignment

The project's main framework is SDG 11. Target 11.2 calls for safe, affordable, accessible and sustainable transport; and Target 11.6 calls for a reduction in the negative environmental per-capita impact of cities, including through addressing air quality and waste management. The University Green Challenge translates these overarching goals into actions on campus. Target 11.2: Walking, cycling and public-transport actions reward lower-impact mobility choices.

- Target 11.6: Recycling, reusable bottles and switching off unused devices support waste and resource efficiency awareness.
- Target 11.7 and community participation: Campus clean-up activities strengthen care for shared and green spaces.

1.2 Project Overview

The system is gamified and has verification. One of six pre-designed eco-actions is chosen and a photo is attached with a description. The request goes into an administrative review queue. Approval adds points and a positive points to the student's trust score; rejection does not award points and subtracts from trust. The points generated, trust progress, and trust rank are displayed on the dashboard and leaderboard, while administrators have access to the aggregated analytics.



Student login screen and University Green Challenge value proposition

The purpose of the challenge is reiterated on the opening screen prior to entering data. It has a concise value statement, examples of eco-actions and clear Student and Admin options to choose from, enabling the intended workflow to be understood instantly. The same green color scheme and the clear action labels are translated into the dashboards, which also help with learnability.

2. Problem Statement

2.1 Real-World Problem

University campuses create opportunities for sustainable action every day, but these happen in small chunks, in an informal manner and often aren't easily quantifiable. A student can recycle, access public transport or participate at a clean-up project without a clear tracking system to record the action, provide a sense of achievement, or provide an understanding of a cumulative number of actions. Administrators can also have lack of a simple view of participation, popular actions and claim outcomes.

The problem isn't that there's not anything that can be done green, it's that there's no process that's light, transparent and motivating that connects action, evidence, verification, reward and feedback. If there is no such process, sustainability initiatives may rely on a single event instead of repeatable actions and data on participation may be challenging to organise.

2.2 Causes and Effects

- Low visibility: students have poor visibility of how small daily actions contribute to a larger campus effort.
- Weak motivation: Acknowledgment or progress feedback for responsible behaviour may not be immediate.
- There is some limited verification: Activities may be self-reported, which can be difficult to believe without evidence and review.
- Fragmented records: there is no history/leaderboard/analytical view of combined activities.
- Administrative workload: Manual review, without filtering, and inefficient status tracking and data persistence.

3. System Objectives and Scope

3.1 Main Objective

To create a usable Python application that encourages university students to take verifiable sustainable actions and offers an administrative structure to review participation and track results.

3.2 Specific Objectives

- Assign students with an individual student ID and set the total number of points on each student record to 0 and assign a trust score of 100.
- Provide students who are logged in with access to submit one of 6 eco-actions – photo and meaningful description.
- Enter all claims in an administrative review queue and give points only if they are approved.
- Keep a trust score which is understandable and will adjust based on accepted and rejected submissions.
- Provide role-appropriate dashboards, claim history, leaderboard ranking and challenge analytics.
- Automatically save and restore records with human-readable JSON file.

3.3 Functional Scope

○ Log in with a registered ID for students, and password for administrators. ○ Admission a new student into the system with a new ID.

- Submit eco-action claim with image supporting evidence and description. ○ Ensure the fields are completed, that images are correct and that the evidence is unique. ○ Approve, reject, mark under review or request more evidence. ○ Update points and trust in accordance to the decision of the review. ○ Show how many points and trust points a leader has. ○ Show student claim history & review status. ○ Calculate total claims, approval rate, rejection count and points awarded.
- Store all the records and pending request identifiers in JSON.

3.4 Quality Requirements

In addition to its main functions, the application should be easy to use, reliable and simple to maintain. Clear navigation and readable messages, students and administrators on separate screens on the interface. Input validation and administrator review helps to ensure that records maintained for students and claims are accurate.

The project is divided into separate modules for the interface, application logic, data models and data storage. This helps code to be more readable and maintainable. The application is installable with the requirements file provided and its records are stored as a JSON file. The challenge process is also visible, as points, trust scores, claim status and ranking results are clearly displayed to users.

4. Computational Thinking Approach

4.1 Decomposition

The big issue of prodding sustainable campus behaviour was split into small components: user access, student sign up, claim entry, evidence validation, review, scoring, trust, ranking, analytics, and persistence. This rendered it more straightforward and each duty was put in place and tested separately.

The project has been broken down into the following smaller parts:

- Access: Determine who the user is and show them the correct screens.
- Capture the selected eco-action, picture and description from the student.
- Validation: Ensure that all the required information is complete and valid.
- Review: Give the administrator permission to approve, reject or continue the review of a claim.

- Reward: Record points and increase student's trust score following review.
- Insight: Set up the leaderboard and work out your challenge analytics.
- Persistence: Save and restore the application data with a JSON file.

4.2 Pattern Recognition

There are repeating patterns in multiple features. The registration process and claim submission are both input-validate-process-save-feedback. The layout of dashboard tables is in the clear-refresh-loop-insert pattern. The claim statuses are mapped to readable labels and colours. Identifying these patterns enabled the use of helper functions to be re-used and to have consistent error messages without repetition.

4.3 Abstraction

The application models only information that is needed for the challenge. Student, Claim and PendingRequest dataclasses hide implementation detail behind clear domain entities. The ChallengeStore provides operations such as register_student(), submit_claim() and review_claim(), so interface code does not need to understand JSON serialization or the internal claim list structure.

4.4 Algorithmic Thinking

Key behaviours are expressed as deterministic step sequences with controlled decisions. The leaderboard sorts students by descending points, descending trust, name and student ID, creating stable results even when scores are tied. Evidence images are read in blocks.

Pseudocode for the main application workflow:

START APPLICATION

Load saved students, claims and pending requests

Display role selection

IF role is Student THEN

Validate registered student ID

Display dashboard, leaderboard and history

Collect eco-action, photo evidence and description

Validate fields and image type

Calculate SHA-256 fingerprint

IF fingerprint was used before THEN reject submission

ELSE create Pending Review request and save data

ELSE IF role is Administrator THEN

Validate administrator password

Register students or select a pending claim

IF claim is approved THEN add points and increase trust

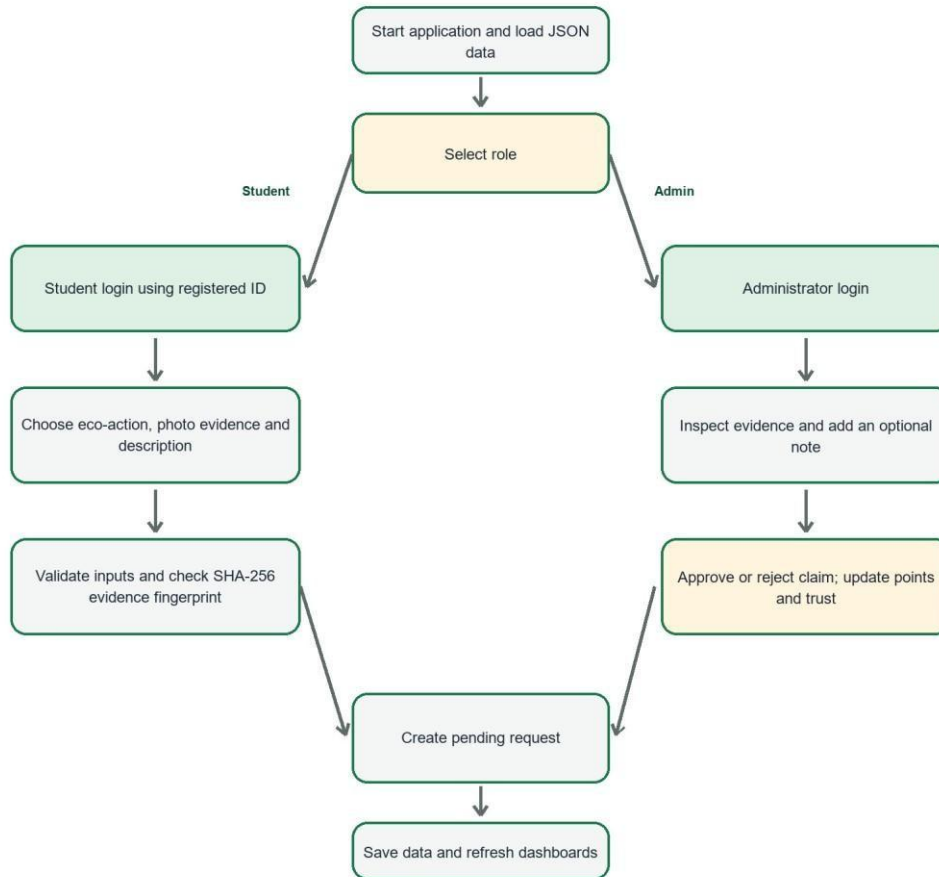
ELSE IF claim is rejected THEN keep points and reduce trust

Save data and refresh all views

END IF

END APPLICATION

Main Operational Flow

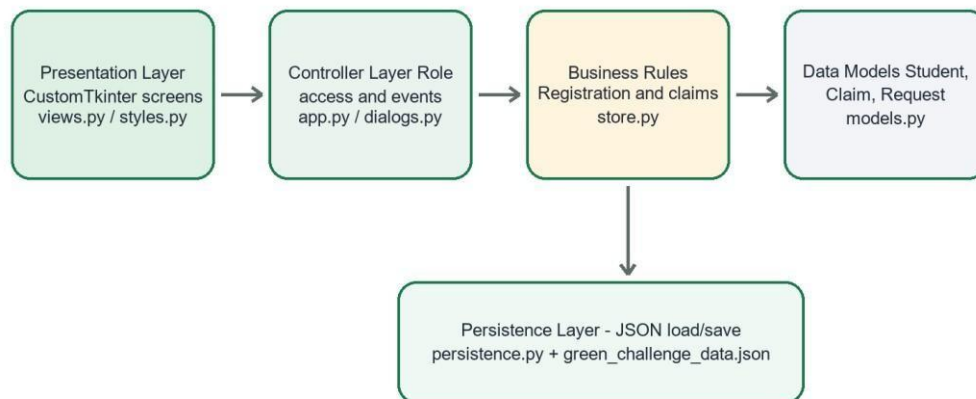


5. Application Design

5.1 Layered Architecture

The design separates presentation from business rules. `views.py` constructs the CustomTkinter screens; `app.py` coordinates sessions, navigation and user events; `store.py` contains registration, claim, trust and leaderboard rules; `models.py` defines the core entities; and `persistence.py` manages JSON input and output. Supporting modules centralize constants, styling and reusable feedback dialogs.

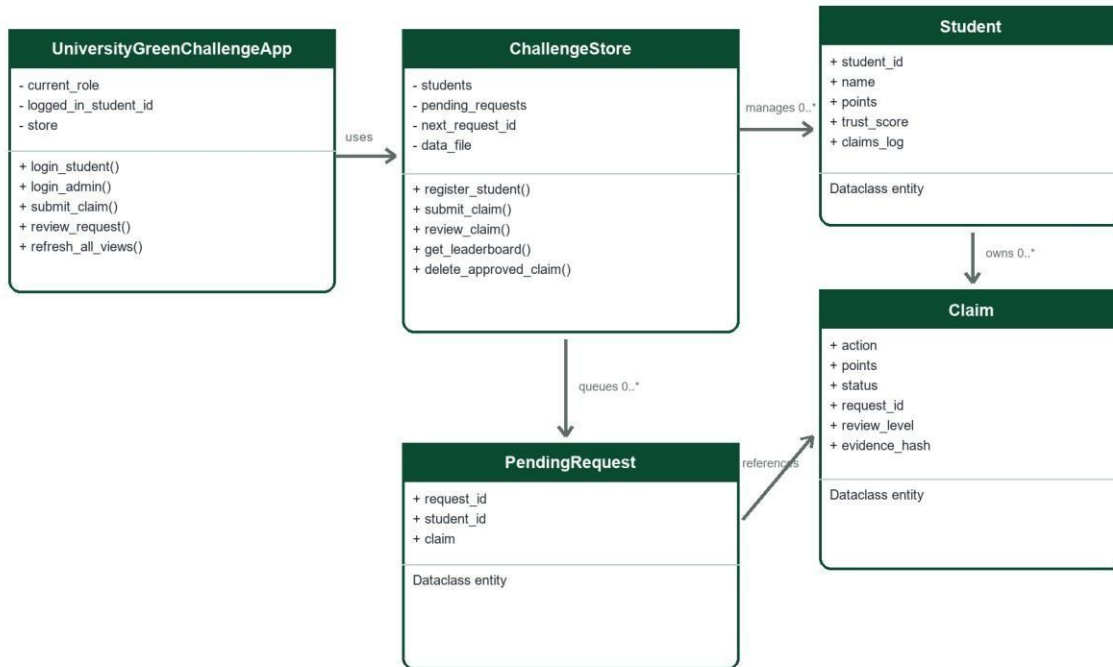
Layered Application Architecture



5.2 UML Class Diagram

A ChallengeStore instance is owned by UniversityGreenChallengeApp and also coordinates all operations that are visible. ChallengeStore stores Student and PendingRequest objects. Each Student has one or more Claim record and each PendingRequest refers to the Claim for which final decision is pending. This relationship helps maintain the responsiveness of the interface, while domain rules remain in one place.

UML Class Diagram



5.3 Data Model

Entity	Main fields	Purpose
Student	student_id, name, points, trust_score, claims_log	Represents one participant and cumulative progress.
Claim	action, points, status, request_id, evidence fields, timestamps, note	Represents one submitted sustainability action.
PendingRequest	request_id, student_id, claim	Links an uncompleted review to its student and claim.

JSON document	students, pending_request_ids, next_request_id	Provides durable local storage across sessions.
---------------	--	---

5.4 Eco-Action Scoring Design

Eco-action	Points after approval
Recycled campus waste	10
Walked or cycled to class	15
Used a reusable bottle	5
Turned off unused devices	8
Joined a campus clean-up	25
Used public transport to campus	12

6. Implementation Details

6.1 Technology Stack

The application was developed using Python as the main programming language. CustomTkinter was used to create the role-based graphical interface, while Tkinter and ttk were used for file dialogs, message boxes,

tables, and chart components. Pillow was used to load and display images. Application records are stored locally in a human-readable JSON file.

6.2 Repository and Module Structure

The repository is available at the following URL and is included directly in the report as required: https://github.com/babaset/University_green_challenge

Module	Responsibility
university_green_challenge.py	Application entry point.
green_challenge/app.py	Controller, authentication, events, refresh and analytics.
green_challenge/views.py	Screens, navigation, forms, cards and tables.
green_challenge/store.py	Core registration, claim, trust and leaderboard rules.
green_challenge/models.py	Student, Claim and PendingRequest dataclasses.
green_challenge/persistence.py	Safe JSON load and save functions.
green_challenge/constants.py	Eco-actions, points, icons and image formats.
green_challenge/styles.py	Shared visual tokens and table styling.

green_challenge/dialogs.py	Reusable feedback dialogs.
----------------------------	----------------------------

6.3 Codebase Metrics and Modularity

Static analysis of the repository identified approximately 2,130 Python source lines, 84 function or method definitions and 6 classes. The code therefore exceeds the minimum function requirement and demonstrates strong modular separation. Representative reusable operations include register_student(), submit_claim(), review_claim(), get_leaderboard(), load_challenge_data(), save_challenge_data(), configure_style() and show_feedback().

6.4 Variables, Operators, Conditions and Loops

- Data types and variables: dictionaries associate actions to points, lists store claims and pending requests, sets are used to prevent repeated processing, strings are used to store identifiers and integers are used to store points and trust.
- Operators: arithmetic to update points and trust, comparison to set score thresholds, logical to chain together validation conditions, and division to determine the approval rate.
- Conditional statements: if, elif and else control authentication, status labels, validation, review decisions and trust-based scrutiny.
- For loops - refresh tables, process students and claims, create navigation items, hash image blocks and calculate analytics.
- Values are input by the user using the CustomTkinter Entry, ComboBox, Textbox and file-dialog controls, and validated by reading the values using .get().Excerpt 1. Dictionary-based eco-action configuration

```
ECO_ACTIONS = {
    "Recycled campus waste": 10,
    "Walked or cycled to class": 15,
    "Used a reusable bottle": 5,
    "Turned off unused devices": 8,
    "Joined a campus clean-up": 25,
    "Used public transport to campus": 12,
}
```

Excerpt 2. Conditional reward and trust logic

```

if decision == "approve":
    student.points += claim.points
    student.trust_score = min(100, student.trust_score + 5)
claim.status = "Admin-Approved" elif decision == "reject":
    student.trust_score = max(0, student.trust_score - 20)
claim.status = "Rejected" else: raise
ValueError("Unknown review decision.")

```

Excerpt 3. Block-based SHA-256 evidence fingerprinting

```

digest = hashlib.sha256() with open(evidence_path, "rb")
as evidence_file: for block in iter(lambda:
evidence_file.read(1024 * 1024), b''):
    digest.update(block) return digest.hexdigest()

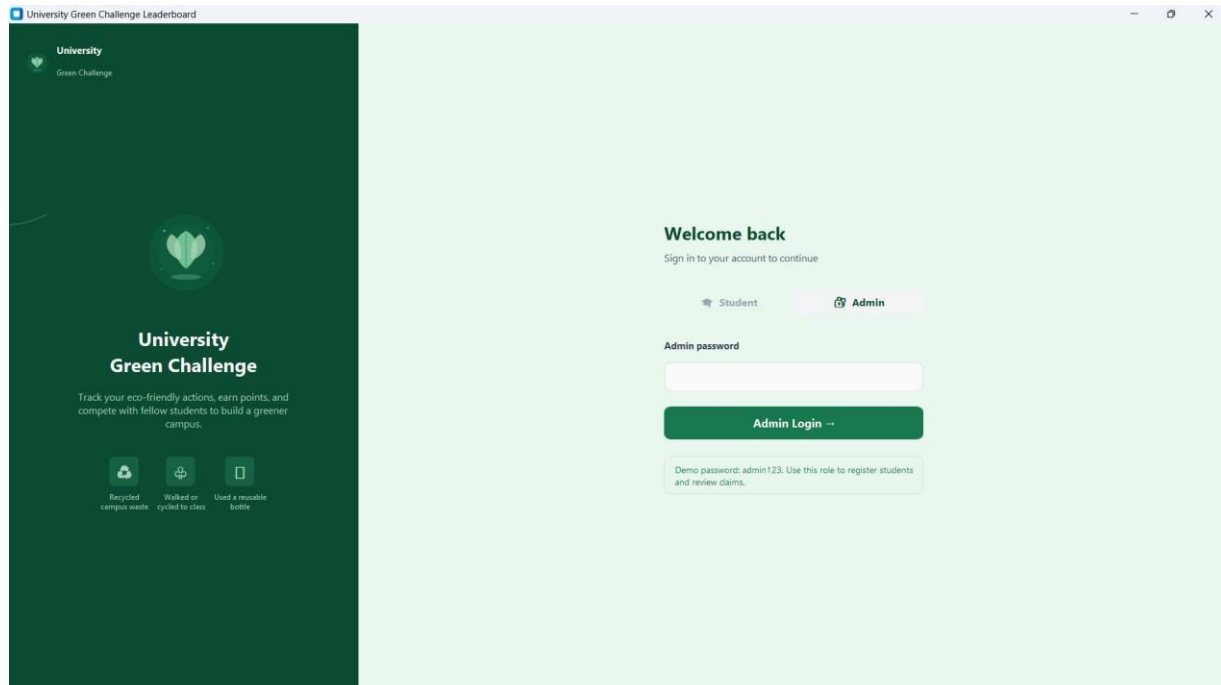
```

6.5 Input Validation and Error Handling

Both interface and business-rule level validation. The interface verifies the active role, the necessary values, the evidence path, the file existence and the supported image extension. The store does not accept blank names, blank IDs, duplicate student IDs, unknown actions, unregistered students or re-used evidence fingerprints. File and JSON operations are surrounded by exception handling to provide users with relevant error messages rather than an unhandled exception.

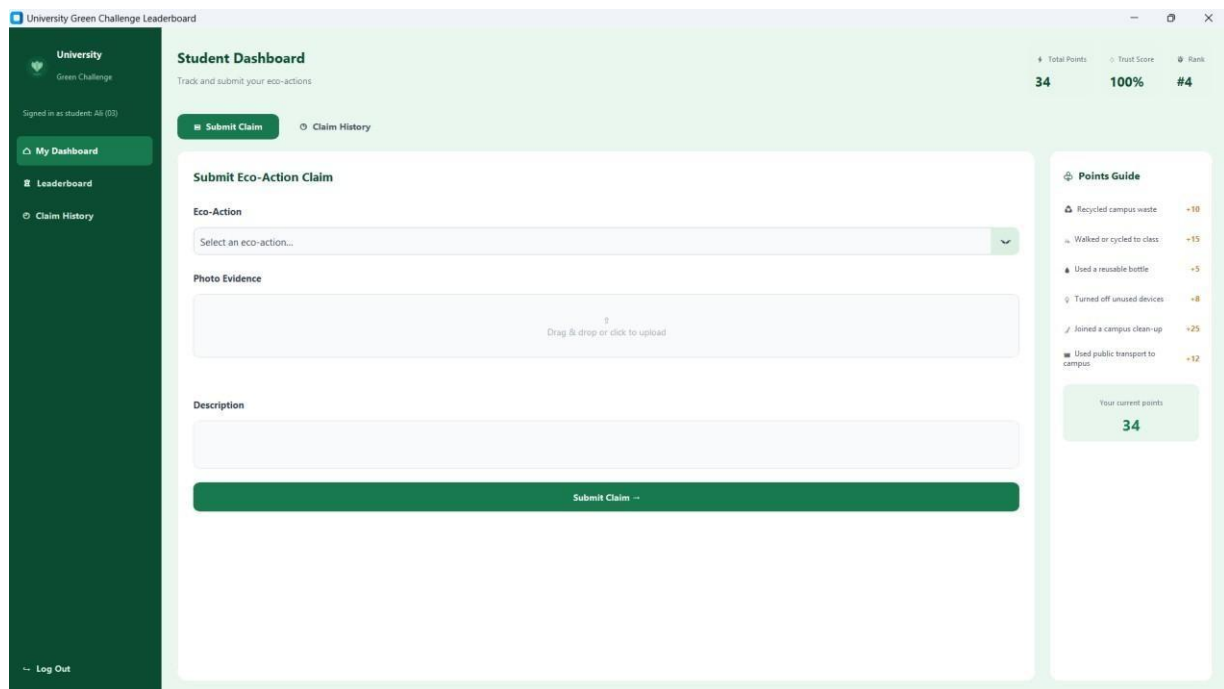
6.6 User Experience and Role-Based Navigation

The visual system of the interface is consistent (all in green), the headings are descriptive, the status colours are clear, and the action labels are clear. Students only view the dashboard, leader board and their own history. Administrators access the registration, review, analytics, leaderboard and history tools. This minimizes navigational complexity and student access to administrative actions.



Administrator login view with role selection

Role Selection is enforced prior to accessing the main workspace. The student enters a registered identifier and the administrator enters the password set up in the application, and the application then builds the appropriate navigation menu, and the functions that are not part of that role are hidden.



Student dashboard with metrics, claim form and points guide

Student Workflow: The student signs in using a registered student ID, selects an eco-action, chooses a supporting photo and enters a short description. After submitting the claim, the student waits for the administrator’s decision and can later view the claim history, points and leaderboard position.

Administrator student-registration screen

7. Testing and Sample Outputs

7.1 Testing Strategy

Testing for combined independent functional checks and also visual evidence of the running desktop application. Core business rules were applied to temporary data files and temporary proof images and no changes were made to the source code or repository data. The end-to-end claim and claim approval workflow in the actual application is shown through the interface screenshots.

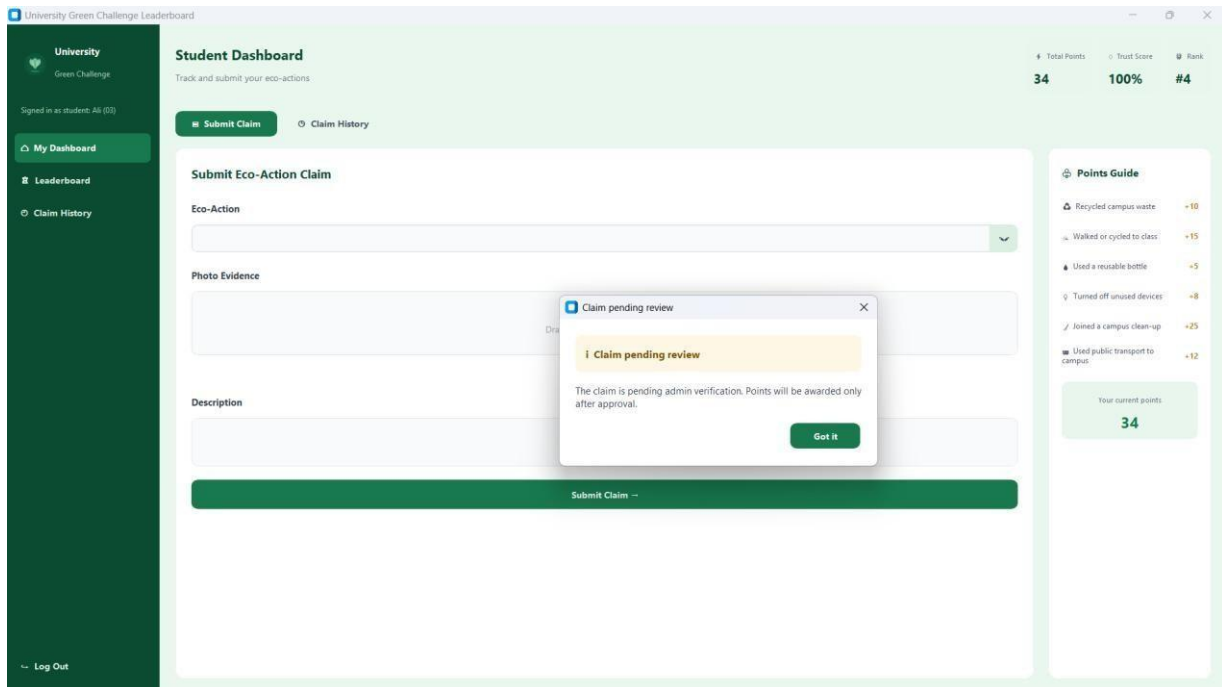
7.2 Functional Test Results

	Test	Input / action	Expected result	Status
1	Register student	Lowercase unique ID	ID normalized and stored	PASS

2	Duplicate registration	Existing student ID	ValueError prevents duplicate	PASS
3	Valid claim	Registered student and new image	Pending claim worth 10 points	PASS
4	Duplicate evidence	Same image for another claim	Submission rejected by hash	PASS
5	Approve claim	Approve pending request	Points added; status approved	PASS
6	Reject claim	Reject at trust score 60	Points unchanged; trust becomes 40	PASS
7	Leaderboard	Students with different scores	Highest points ranked first	PASS
8	Persistence	Reload saved temporary JSON	Points and claim history restored	PASS

7.3 Sample Output: Student Submission

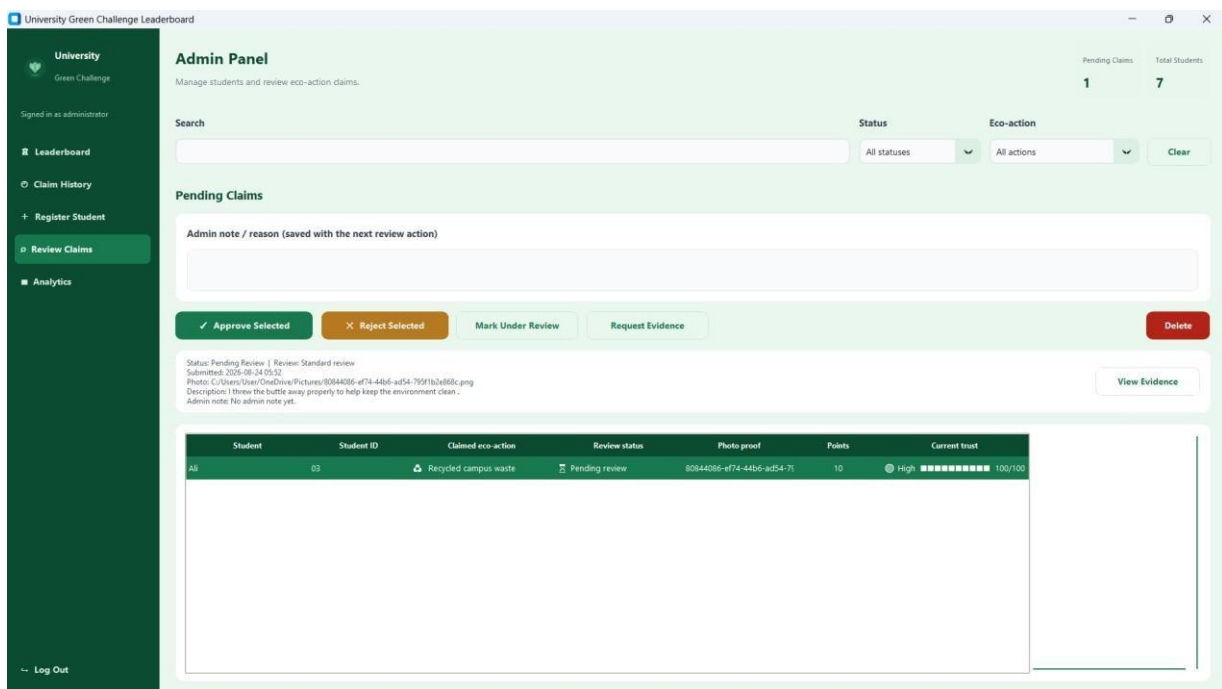
After a student sends in an eco-action claim the screen shows that the request is waiting for the administrator to check it. It also says that points will only be given once its approved. This message makes sure the student doesn't think the score changes just because they submitted something.



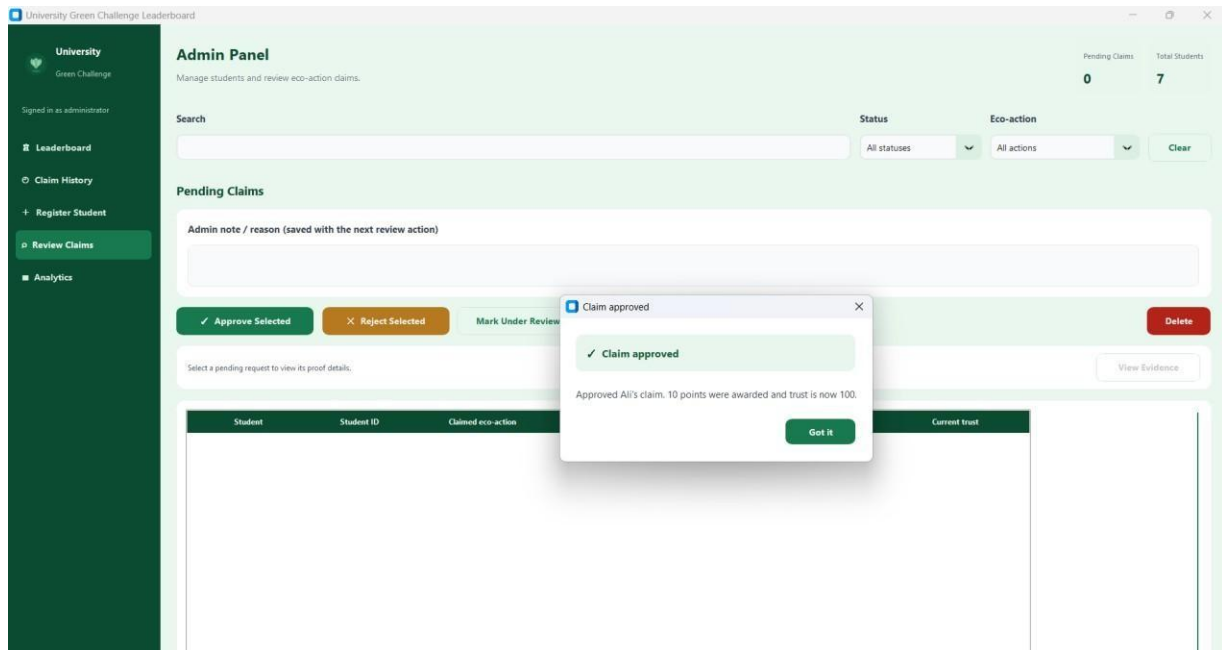
Successful submission enters the pending-review state

7.4 Sample Output: Administrator Review

In the review queue you can see the student, ID claimed action, current status, proof filename, points and trust progress. If you select a row you will see the submission details. This allows the administrator to view evidence approve, reject or request evidence.



Administrator Workflow: Administrator logs in. Administrator adds students. Looks at claims that have been sent in. The administrator checks the details of each claim and the photo that goes with it. After that the administrator makes a decision, whether to allow or not the request. After that the administrator can see the points, the status of each claim the list of top performers and the numbers and information about the activity, from the administrator panel.

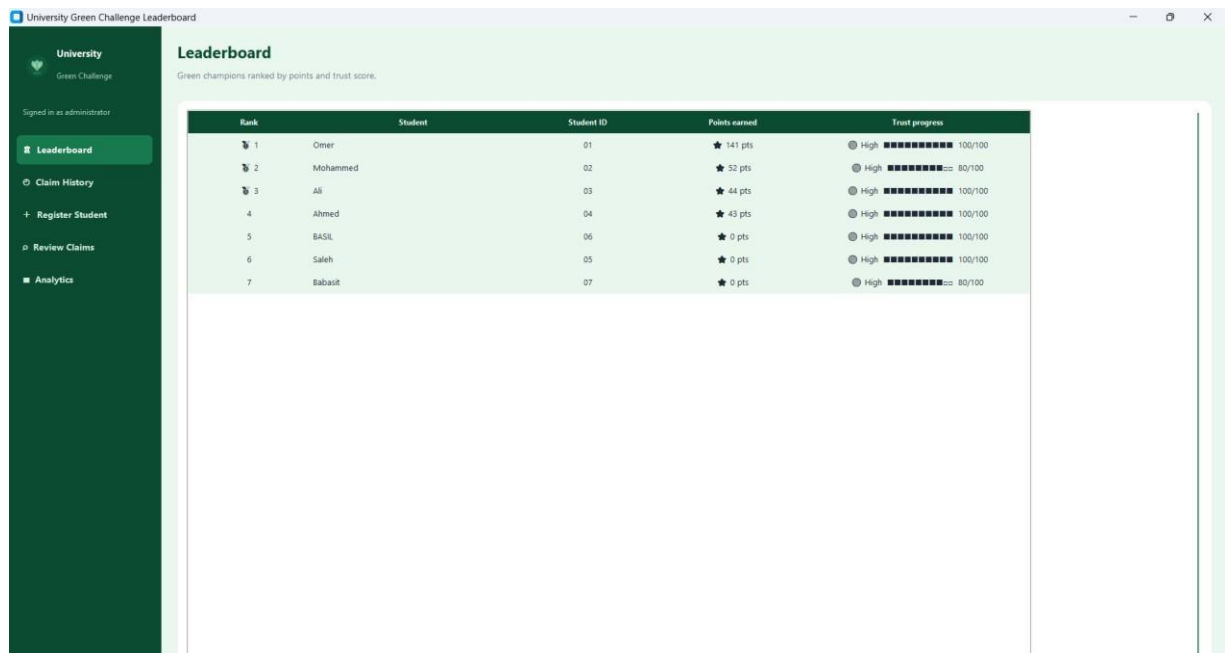


Claim approval confirmation with awarded points

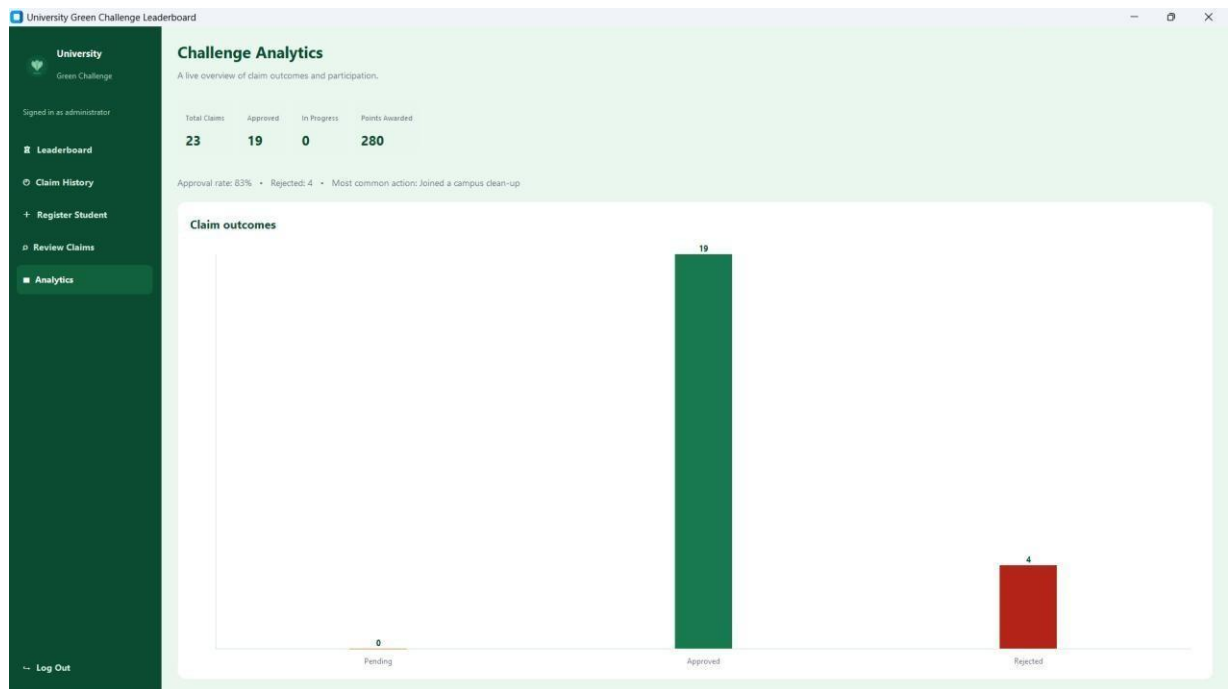
If the claim is accepted, then it is removed from the pending list and the points are granted based on the configuration and the student's trust score is displayed. The below confirmation gives immediate and specific feedback regarding the outcome.

7.5 Sample Output: Leaderboard and Analytics

The leader board is based on points first, trust second; name and ID are used as tie breakers. This will make the ranking reproducible and comprehensible. The analytics view will provide a summary of total claims, total approvals, work in progress, awarded points, approval ratio, rejected claims and the top eco-action.



Leaderboard ranked by points and trust progress



Challenge analytics dashboard and claim-outcome chart

Observed analytical output: The supplied capture shows 23 total claims, 19 approvals, 4 rejections, an 83 percent approval rate and 280 awarded points. These values are sample project data, not a general claim about all university students.

8. Discussion and Limitations

8.1 Achievement of Objectives

Maintaining the connection between sustainable action and evidence, administrative verification, points and visible progress is at the core of the application. Student workflow is short enough to be repeated and administrator gets controlled registration, filtering and status changes/review. The leaderboard and analytics translate the individual activity into collective feedback that can be used to motivate and monitor campaigns.

Computational thinking is also applied in the implementation. The module boundaries show decomposition; the domain dataclasses and the ChallengeStore interface show abstraction; the screen/validation helpers are examples of reusable pattern recognition; and the hashing, review, trust, ranking and analytics are examples of algorithmic thinking.

8.2 Strengths

- Distinct division of interface, controller, business rules, data models and persistence.
- Usability: Role based navigation that only exposes relevant actions to each user type.
- Evidence integrity control done using the SHA-256 fingerprints instead of file names.
- Transparency of action points, review states, trust feedback and leaderboard ordering.
- Local persistence and recovery after application restart – automatic.
- Created visual analytics to help administrators gain insight into participation results.

8.3 Current Limitations

- No data is synced between application devices; it is stored only in one local JSON file.
- Evidence records contain local file paths, which can mean that the image cannot be found in the evidence if it is moved or deleted.
- The administrator password is not stored securely hashed, but is instead a demonstration constant.
- SHA-256 stops the reuse of the same bytes in images but is not a guarantee that the content is the same as the action being taken.
- The extra-scrutiny decision also incorporates randomness, which can result in varying levels of scrutiny being given to similar trust scores.
- The prototype includes automated business-rule checks, but not an automated graphical-interface test suite or a formal user study.

8.4 Future Improvements

- Move to a managed Database for local JSON and Multi User authenticated service.
- Store copies of evidence in application managed storage instead of external local storage paths.
- Add in secure hashing for admin accounts and access to roles.
- Include campaigns, badges, team challenges and reward redemption that can be configured.

- Write automated UI tests, and perform an actual campus usability study.
- Provide monthly sustainability reporting to university management and SDG reporting.

9. Conclusion

By showing how to create a small, but well-coordinated system of campus sustainability, University Green Challenge proves that a focused Python application can make a big difference. Environmental awareness and visible participation is the real world application where the solution fills the gap with action recording and photographic evidence, administration review, points, trust, ranking and analytics.

The project is directly connected to SDG 11 - Sustainable cities and communities, specifically sustainable mobility and minimizing the environmental impact of urban areas. The modular design and the thousands of user-defined functions, input validation, loops, conditions, persistence and evidence hashing exemplifies the programming and computational-thinking concepts covered in the course. Central business rules were tested and captured screens show a complete operational workflow from logon to approval and analysis reporting.

This is a local prototype of the new version, however it is a useful base for creating a larger platform for the green-campus. Most importantly, it makes it easier to record sustainable behaviour, easier to be believed to have done it, and easier to do it again!

Final result: A sustainable challenge that is functional, modular and easy to use so that people can become more sustainable with their daily actions and get recognized for it on the campus.

References

United Nations. (2025). Goal 11: Sustainable cities and communities. United Nations Department of Economic and Social Affairs. <https://sdgs.un.org/goals/goal11>

United Nations Statistics Division. (2026). Sustainable Development Goal 11: Build inclusive, safe and sustainable cities and human settlements. <https://unstats.un.org/sdgs/report/2026/Goal-11/>

Python Software Foundation. (2026). Python 3 documentation. <https://docs.python.org/3/>